

Nástroj pre manažment variability založený na aspektoch v rámcoch

Leon Radó

Fakulta informatiky a informačných technológií

Slovenská technická univerzita v Bratislave

Bratislava, Slovenská Republika

xrado@stuba.sk

Abstrakt—So zvyšujúcim sa počtom variabilných častí kódu a ich závislosťou sa manuálny manažment variability stáva čoraz náročnejším a menej udržateľným, preto je dôležité riešiť nástrojovú podporu pre jeho efektívnu správu. Jedným z možných prístupov, ako riešiť tento problém je pomocou technológie aspektov v rámcoch (FA), teda kombináciou aspektovo-orientovaného programovania (AOP) a technológie rámcov, ktorá zaručuje vysokú konfigurovateľnosť a nezávislosť od programovacieho jazyka implementácie. V momentálnej dobe však chýba efektívna podpora, ktorá by zohľadňovala aktuálne technológie a aspekty v rámcoch, čo vedie k ich obmedzenému využívaniu pri tvorbe nového softvéru. Výsledkom práce je návrh a vyhodnotenie metodológie vývoja zahŕňajúcej technologické inovácie pre manažment variability v zdrojovom kóde na základe aspektov v rámcoch, ktorá je podporená nástrojom vo forme jednostránkovej aplikácie (SPA). Použitelnosť a efektivita implementovaného nástroja sú vyhodnotené overením hypotéz odvodených z navrhnutej metodológie pomocou metrik času, počtu krokov a hodnotenia na Likertových škálach. Navrhnuté riešenie štatisticky významne zlepšilo riešenie úloh súvisiacich s konfiguráciou a odvodením výrobkov a úpravou variabilného kódu, avšak pri orientácii v projekte nepreukázalo štatisticky významný prínos v porovnaní s pokročilým celotextovým vyhľadávaním v bežnom vývojovom prostredí. Subjektívne hodnotenie použiteľnosti nástroja dosahovalo v priemere úroveň 4, teda skôr súhlasím.

Kľúčové slová—rad softvérových výrobkov, aspekty v rámcoch, manažment variability, odvodenie výrobku, aspektovo-orientované programovanie

I. ÚVOD

Rady softvérových výrobkov (SPL) poskytujú systematický spôsob na dosiahnutie znovupoužitelnosti vynechaním fázy analýzy pre každý softvérový výrobok zvlášť, pričom každý z nich sa spravidla odvodzuje z jedného zdrojového kódu pomocou definovaných pravidiel. Tento princíp umožňuje mať jeden zdrojový kód a na základe zvolených funkcionalít a nastavení v rámci tohto kódu odvodzovať viacero softvérových výrobkov, ktoré postačujú požiadavkám zákazníkov a skracujú čas uvedenia na trh [1]. Realizovateľný je pomocou aspektov v rámcoch (FA), teda kombináciou technológie rámcov a aspektovo-orientovaného programovania (AOP). Technológia rámcov umožňuje na základe zvolených premenných a anotácií označiť variabilné časti kódu a zabezpečiť vysokú konfigurovateľnosť, pričom využíva XML (*Extensible Markup Language*) anotácie, čo zaručuje jej nezávislosť od programovacieho jazyka [2]. Na druhej strane aspektovo-orientované

programovanie umožňuje oddeľovať prierezové funkcionality (napr. logovanie) od hlavnej logiky aplikácie [3].

Napriek potenciálu tohto konceptu si technológia aspektov v rámcoch nezískala širšie uplatnenie a v dnešných softvérových projektoch sa s ňou spravidla nestretáme. Čiastočne za to môže aj úpadok AOP, ktoré sa prestalo využívať kvôli zložitosti ladenia, náročnej údržbe a ďalším praktickým problémom vo veľkých systémoch [4]. Keďže tento spôsob vznikol iba ako koncept pred viac ako 20 rokmi, v dnešnej dobe chýba efektívna podpora, ktorá by zohľadňovala aktuálne technológie a umožňovala by vývoj pomocou aspektov v rámcoch v plnej miere. Tiež treba podotknúť, že tradičné prístupy k správe variability, ako napríklad jej riešenie pomocou komentárov, direktív preprocesora, značiek a pod. môžu znižovať zrozumiteľnosť kódu [5], čo potom vedie k zvýšeným nákladom na údržbu a sťažuje možnosti ďalších úprav v priebehu životného cyklu daného softvéru.

Výsledkom práce je návrh a vyhodnotenie metodológie vývoja zahŕňajúcej technologické inovácie pre manažment variability v zdrojovom kóde založenom na aspektoch v rámcoch. Metodológia je podporená nástrojom pre manažment variability, ktorý umožňuje spracovanie projektu s anotovaným zdrojovým kódom, správu konfigurácie variability, kontrolu konzistencie a odvodzovanie výsledných softvérových výrobkov, spolu s ďalšími podpornými funkcionalitami, ktoré uľahčujú a urýchľujú prácu vývojárov. Zdrojový kód nástroja spolu s krátkym používateľským manuálom v anglickom jazyku je dostupný na nasledujúcom odkaze.¹

Sekcia II opisuje relevantnú literatúru a existujúce prístupy k manažmentu variability spolu s ich výzvami. Sekcia III predstavuje navrhnutú metodológiu, inovácie pre správu variability a nástroj spolu s jeho funkcionalitami. Sekcia IV vyhodnocuje efektivitu metodológie a nástroja pomocou scenárov, hypotéz a subjektívneho hodnotenia účastníkov testovania. Sekcia V analyzuje obmedzenia riešenia spolu s možnými vylepšeniami. Sekcia VI sumarizuje prácu a dôležité zistenia.

II. ANALÝZA

V dnešnej dobe, kedy je tvorba softvérových výrobkov čoraz dostupnejšia a rozšírenejšia, viacerým zákazníkom môže

¹<https://doi.org/10.6084/m9.figshare.31420682>

postačovať softvér, ktorý obsahuje rovnaké základné funkcionality, len s čiastočnými úpravami prispôbenými ich špecifickým potrebám [6]. Každý rad softvérových výrobkov pozostáva z voliteľných a povinných funkcionalít, ktoré na seba môžu alebo nemusia nadväzovať, preto je dôležité vedieť efektívne manažovať variabilitu voliteľných funkcionalít. S rastúcim množstvom variabilných funkcionalít sa však zvyšuje aj náročnosť pri udržovaní kódu v čitateľnom a modulárnom stave [7].

Riešenie variability v radoch softvérových výrobkov pomocou samotnej technológie rámcov síce zaručuje podporu správy variability a vysokú konfigurovateľnosť [8], ale neumožňuje oddelenie prierezových funkcionalít, ktoré sa v kóde častokrát nachádzajú na viacerých miestach. Aj keď takéto riešenie variability poskytuje obrovské výhody vďaka vysokej parametrizácii, prichádza aj s vážnymi komplikáciami pri údržbe softvéru, keďže zmeny prierezových funkcionalít je potrebné vykonať súčasne na viacerých miestach. Jedným z možných prístupov ako riešiť tento problém je pomocou technológie aspektov v rámcoch, teda kombináciou aspektovo-orientovaného programovania a technológie rámcov, ktoré zaručujú vysokú variabilitu kódu pomocou XML anotovania voliteľných funkcionalít, nezávisle od programovacieho jazyka implementácie, nastavovania hodnôt premenných a oddelenia prierezových funkcionalít [2], [3].

Konceptu aspektov v rámcoch sa venovali najmä Loughran a Rashid (2004). Navrhli spôsob ako oddeliť jednotlivé funkcionality kódu do rámcov (modulov) pomocou XML anotácií a pomocou vysokej parametrizácie umožnili aj konfiguráciu aspektov [2], [3]. Označovanie rámcov, ktoré nazvali *Lancaster Frame Processor (LFP) Meta Language*, vzniklo z XML anotácií, ktoré boli čiastočne prevzaté z tradičných rámcových technológií ako je *XVCL (XML-based Variant Configuration Language)* a boli doplnené o niektoré ďalšie vlastnosti [3].

Správa variability v radoch softvérových výrobkov bola v priebehu rokov skúmaná z viacerých pohľadov. Väčšina tejto práce sa zaoberá tradičnými metódami v rámci *SPL*, ktoré sa sústreďujú na konfiguráciu softvérových výrobkov na základe komentárov, direktív preprocesora, značiek a pod. odvodených od modelu vlastností [9]. Model vlastností poskytuje prehľad o všetkých povinných a voliteľných funkcionalitách, ktoré zobrazuje pomocou stromovej štruktúry [10]. Problémy takýchto riešení zdôrazňovali napríklad Bosch (2001) a Jaring (2002), kde upresnili, že zložitosť variability stúpa s počtom rozličných funkcionalít a kvôli nedostatku nástrojov, ktoré by tento prístup zohľadňovali, manuálne manažovanie variability sa stáva nezvládnutelné [7], [9]. Zdôrazňujú potrebu nástroja s podporou pre správu komplexnej variability kódu. Ako existujúci nástroj možno uviesť *FeatureMapper* (2008), ktorý umožňuje mapovať funkcionality na prvky reprezentujúce ich realizáciu [11]. Pri tvorbe *SPL* sa spomína aj aspektovo-orientované programovanie, ktoré malo adresovať problémy so zložitou pri škálovaní softvéru. Keďže jedna vlastnosť môže pokojne korešpondovať s viacerými komponentmi, práve *AOP* dokáže oddeliť tieto pretínajúce sa funkcionality [12]. Figueiredo (2008) skúmal *AOP* pri vývoji *SPL* so

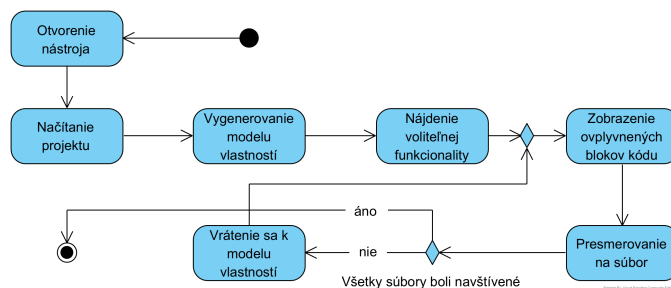
zistením, že tento prístup podporuje modularitu a celkový návrh softvéru, no na druhej strane prichádza s problémami, napríklad pri zmene povinných funkcionalít na voliteľné [13].

Zhrnutím vyššie spomenutého je možné povedať, že existuje viacero spôsobov ako riešiť vývoj *SPL*, ktoré sa sústreďujú na dôležitosť modularity a variability, no väčšina z nich trpí zložitou pri škálovaní softvéru alebo nedostatočnou podporou nástrojov, ktoré by pomáhali programátorom pri vývoji a odvodzovaní výrobkov [14], [15]. Tiež treba spomenúť, že väčšina zdrojov je viac ako 20 rokov stará, preto je potrebné vyvinúť nástroj, ktorý by zohľadňoval aktuálne technológie a postupy pri vývoji moderného softvéru.

III. RIEŠENIE

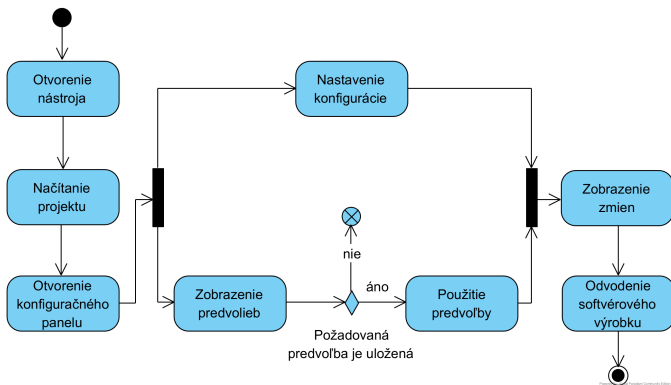
Navrhnutá metodológia vývoja spolu s nástrojom a jeho podpornými funkcionalitami umožňuje vývojárom vykonávať hlavný postup práce pre správu variability v radoch softvérových výrobkov. Nástroj poskytuje vývojárom jednoduchšie a použiteľnejšie prostredie pre efektívnu správu variability v projektoch, ktoré využívajú technológiu aspektov v rámcoch. Konkrétne podporuje XML anotácie využívané v *LFP* technológii a teda je postavený na jej princípoch. Metódy vývoja opisujú základné postupy pre bežné úlohy pri správe variability v radoch softvérových výrobkov. Každá metóda sa skladá z krokov, ktoré v celku predstavujú hlavný a kompletný postup práce. Jednotlivé metódy sú definované nasledovne:

- **M1 - Orientácia v projekte:** Cieľom metódy je zlepšiť orientáciu v rozsiahlejších projektoch a urýchliť nájdenie súborov a blokov kódu súvisiacich so zvolenou variabilnou funkcionalitou. Metóda sa používa v situácii, keď používateľ nevie, kde je funkcionalita realizovaná alebo chce pochopiť závislosti medzi variabilnými a spoločnými časťami kódu, ako je znázornené v diagrame aktivít na Obr. 1.



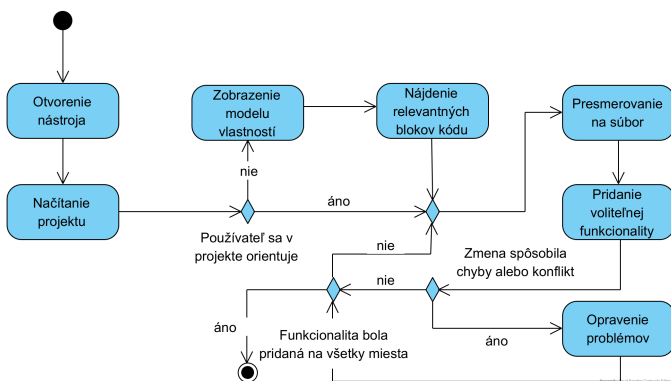
Obr. 1. Používateľský scenár orientácie v projekte

- **M2 - Konfigurácia a odvodenie:** Cieľom metódy je vytvoriť špecifikovaný softvérový výrobok výberom voliteľných funkcionalít a nastavením premenných alebo načítaním konfigurácie z predvoľby a následne overiť správnosť konfigurácie a odvodiť výsledný zdrojový kód. Metóda sa používa pre dodanie výrobku ďalším aktérom. Tento scenár je znázornený na Obr. 2.
- **M3 - Úprava variabilného kódu:** Cieľom metódy je umožniť upravovanie existujúcich voliteľných funkcionalít, pridávanie nových funkcionalít a manipulovať s



Obr. 2. Používateľský scenár konfigurácie a odvodenia

premennými tak, aby sa minimalizovali chyby a konflikty v konfigurácii. Metóda sa používa pri škálovaní radu softvérových výrobkov. Obr. 3 zobrazuje tento scenár.



Obr. 3. Používateľský scenár úpravy variabilného kódu

A. Inovácie v správe variability

V navrhnutom nástroji bola vytvorená podpora pre existujúcu funkcionality na vykonávanie kontroly integrity v *SPL*, podobne ako v [16], generovanie dokumentácie z anotovaného zdrojového kódu a jeho variantov, podobne ako v [17], ale na základe *XML* značiek vyskytujúcich sa v kóde a odvodzovanie finálnych výrobkov podľa konfigurácie, tak ako to robí pôvodný *Lancaster frame processor*. Tieto funkcionality sú zhrnuté v Tabuľke I.

Okrem toho boli navrhnuté nové funkcionality pre správu variability s cieľom zohľadniť technologický pokrok, keďže technológia aspektov v rámci je v súčasnosti zastaraná. Prehľad navrhnutých inovatívnych funkcionalít a ich aplikácie je zhrnutý v Tabuľke II. Hlavným prínosom je správa variability v *HTML* súboroch, ktoré sú priamo vykresľované modernými webovými prehliadačmi alebo parsované pomocou *XML* knižníc v *SPA* aplikáciách. Následne je možné nad nimi komplexne vykonávať dopyty na požadované body variability pomocou kaskádových štýlov alebo *web scraping* knižníc. Taktiež moderné prehliadače umožňujú perzistenciu a načí-

tavanie konfigurácií a systematické prepojenie súvisiacich variabilných častí kódu nachádzajúcich sa na rôznych stránkach.

B. Funkcionalita

Táto časť popisuje implementované funkcionality navrhnutého nástroja. Hlavnou funkcionality riešenia je spracovanie projektu s *XML* anotovaným zdrojovým kódom, ktoré zahŕňa načítanie zvoleného adresára a následne jeho zparsovanie a transformáciu obsahu do objektov konkrétneho typu. Konceptuálne je táto vytvorená reprezentácia podobná abstraktnému syntaktickému stromu (*AST*), ktorý zachytáva hierarchiu a syntaktické vzťahy kódu a umožňuje presné mapovanie závislostí [18]. Pre naše účely je takáto podrobná reprezentácia zbytočná a postačuje rozdeliť kód do blokov podľa typu, pričom sa rozlišuje medzi štandardným kódom, voliteľnou funkcionality, variabilnými premennými a podobne. Výsledná interná štruktúra je uchovaná v pamäti a zostáva dostupná počas celej doby používania nástroja, ako je znázornené na Obr. 4. Okrem toho, nástroj umožňuje vykonávať podporné funkcionality, ktorých podrobnejší popis je uvedený nižšie:

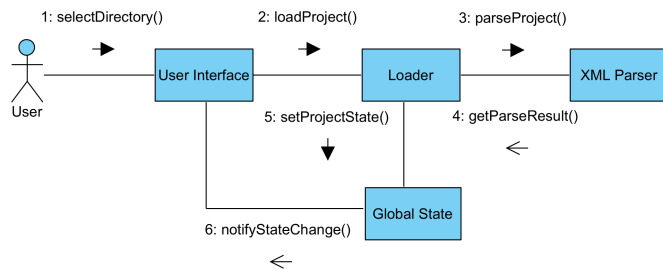
- **Úprava kódu a variabilných častí** umožňuje opravovanie chýb a vykonávanie dodatočných úprav softvéru pomocou textového editora. Taktiež je možné pridávať, odstraňovať alebo upravovať variabilné časti kódu a definovať premenné.
- **Vizualizácia modelu vlastností** zabezpečuje generovanie a zobrazenie modelu vlastností a to automaticky na základe definícií a vnárania jednotlivých funkcionalít a modulov v projekte. Pri kliknutí na určitý uzol v tomto modeli prebehne presmerovanie na súbor, kde je implementovaný.
- **Správa konfigurácie variability** umožňuje aktivovanie alebo deaktivovanie voliteľných funkcionalít, nastavovanie premenných a upravovanie obmedzení. Zmeny sa aplikujú automaticky a konfigurácie je možné ukladať ako predvoľby.
- **Validácia a kontrola integrity** overuje správnosť názvov, atribútov a vnárania *XML* značiek a identifikuje konflikty v konfigurácii. Cieľom je predchádzať chybám, ktoré vznikajú pri manuálnej manipulácii s projektom.
- **Generovanie dokumentácie** umožňuje generovanie dokumentácie zo zdrojového kódu na základe aktuálnej konfigurácie a obsahuje prehľad aktivovaných funkcionalít, hodnoty premenných a ich obmedzení, spolu s ich referenciami na súbory, v ktorých sú implementované.
- **Ukladanie zmien a odvodzovanie výrobkov** zabezpečuje ukladanie projektu so zmenenou konfiguráciou, takže sa dá podľa potreby vytvoriť viacero verzií projektu a ďalej na nich stavať a odvodzovanie výsledných softvérových výrobkov.
- **Prepojenie variabilných častí kódu** má na starosti zjednodušenie orientácie v projekte tým, že variabilné časti kódu sa vedú odkazovať na súbory a moduly, ktoré sú nimi ovplyvnené.

Tabuľka I
EXISTUJÚCE FUNKCIONALITY PRE SPRÁVU VARIABILITY

Podpora správy variability	Funkcia	Zdroj	Technológie
Validácia XML a kontrola integrity	Overuje, či je XML správne formátované, zodpovedá príslušnej XML schéme a je v súlade s obmedzeniami	feature-schema [16]	LFP Meta Language, XML Validator
Generovanie dokumentácie	Poskytuje prehľad o rade softvérových výrobkov a detaily konkrétnych variantov	FeatureIDE [17]	XML Parser, Blob API, RegExp
Odvodzovanie finálnych výrobkov	Generovanie výsledného syntakticky správneho a spustiteľného zdrojového kódu	Lancaster frame processor a jeho AOP rozšírenie	Blob API, State Management, AOP

Tabuľka II
NAVRHNUTÉ FUNKCIONALITY PRE SPRÁVU VARIABILITY S VYUŽITÍM TECHNOLOGICKÝCH INOVÁCIÍ

Podpora správy variability	Funkcia	Inovácie	Technológie
Prepojenie variabilných častí kódu	Zvýraznenie a prepojenie variabilných častí kódu so súvisiacimi súbormi a modulmi	Vizualizácia variability prostredníctvom webových stránok s využitím dynamických UI komponentov	Block-Level Analysis, UI Component Library, SPA Routing
Komplexná správa variability založená na kaskádových štýloch	Uplatnenie kaskádových štýlov na dopytovanie bodov variability	Zjednodušenie dopytov na miesta ovplyvnené variabilnými časťami kódu využitím CSS alebo web scraping knižníc	CSS, SASS, SoupSieve, BeautifulSoup
Automatické generovanie modelu vlastností	Využitie presmerovania na stránky v grafických dynamických komponentoch	Zobrazenie interaktívneho modelu vlastností na základe výskytu variability v kóde	XML Parser, UI Visualization Library, SPA Routing
Úprava zdrojového kódu s variabilnými časťami	Pridávanie nových a upravovanie existujúcich variabilných funkcionalít a premenných pomocou editora	Premietnutie zmien vykonaných cez používateľské rozhranie do anotovaného projektu s priebežnou kontrolou obmedzení	Conditional Rendering, RegExp, XML Parser
Ukladanie zmien v projekte	Uloženie nakonfigurovanej verzie projektu nahradením hodnôt v XML anotáciách	Aktualizácia zdrojových súborov projektu z internej reprezentácie do korektných XML dokumentov	File System API, State Management



Obr. 4. Komunikačný diagram interakcií objektov pri spracovaní projektu

C. Technológie

Používateľské rozhranie nástroja je realizované ako jednostránková aplikácia (SPA). Výhodou tohto prístupu je, že pri interakcii používateľa nedochádza k opätovnému načítaniu celej stránky, ale používateľské rozhranie sa aktualizuje dynamicky, čo zvyšuje jeho reaktivitu na zmeny. SPA prístup je zároveň široko podporovaný modernými *JavaScript* knižnicami, ktoré zlepšujú údržbu, modularitu a interaktivitu používateľského rozhrania [19].

Tento prístup zároveň umožňuje využiť moderné webové technológie na strane klienta. Ukladanie a zdieľanie konfigurácie v aplikácii je riešené prostredníctvom globálneho stavu a perzistencia predvolieb konfigurácie je realizovaná pomocou *IndexedDB*. Ukladanie a načítavanie projektu a export dokumentov sú zabezpečené technológiami *File System API* a *Blob API*, čo umožňuje pracovať so súbormi priamo z prostredia prehliadača. Vďaka SPA prístupu je zároveň možné využiť široké spektrum *UI* knižníc založených na dynamic-

kých komponentoch, čo zjednodušuje tvorbu interaktívneho a estetického používateľského rozhrania.

Riešenie sa zameriava výlučne na technológiu aspektov v rámci a je navrhnuté s dôrazom na nezávislosť od programovacieho jazyka, preto podporuje načítanie projektov implementovaných v ľubovoľnom jazyku. Nástroj je dostupný ako webová aplikácia spustiteľná v prehliadači, teda nevyžaduje inštaláciu a umožňuje používanie v rôznych prostrediach. Zároveň využíva najmodernejšie technológie, čo zabezpečuje rýchlejšiu odozvu a lepšiu použiteľnosť v porovnaní so staršími riešeniami. Pre zachovanie pocitu natívnej aplikácie, ako to býva štandardom pri iných vývojových prostrediach, bola aplikácia v poslednom kroku transformovaná na progresívnu webovú aplikáciu (PWA).

IV. EVALUÁCIA

Na evaluáciu navrhutej metodológie a nástroja bola realizovaná porovnávacia štúdia medzi dvoma pracovnými postupmi, a to správou variability pomocou navrhnutého nástroja a manuálnou správou variability v bežnom vývojovom prostredí. Existujúce nástroje pre manažment variability v radoch softvérových výrobkov neboli pri testovaní použité, pretože aj keď často poskytujú podobnú alebo dokonca pokročilejšiu funkcionalitu, nie sú postavené na technológii aspektov v rámci a typicky nemajú explicitnú podporu pre jazykovo nezávislé XML anotácie.

Hypotézy na otestovanie metodológie sú odvodené z definovaných metód (sú uvedené v poradí H1 – M1 atď.):

- **H1:** Metóda zníži čas a počet krokov navigácie potrebných na nájdenie všetkých blokov kódu ovplyvnených zvolenou voliteľnou funkcionalitou.

- **H2:** Metóda zníži čas a počet krokov navigácie potrebných na nastavenie cieľovej konfigurácie projektu a priebežnú kontrolu obmedzení.
- **H3:** Metóda zníži čas a počet krokov navigácie potrebných na pridanie variabilnej funkcionality a definovanie premennej v konfigurácii.

Hypotézy boli overené na demo projekte *edge observability gateway SPL* implementovaného v jazyku *Java*, ktorý obsahuje viac ako 30 zdrojových súborov s *XML* anotáciami. Projekt zahŕňa viacero voliteľných funkcionalít a premenných, vďaka čomu je vhodný na testovanie navigácie, konfigurácie a upravovania variability v rámci oboch porovnávaných postupov. Testovania sa zúčastnilo 35 vývojárov, ktorí vykonávali 3 úlohy definované v hypotézach, najprv v navrhnutom nástroji a potom v bežnom vývojovom prostredí podľa ich výberu. Neskôr tiež vyjadrili subjektívne hodnotenie spokojnosti na Likertových škálach.

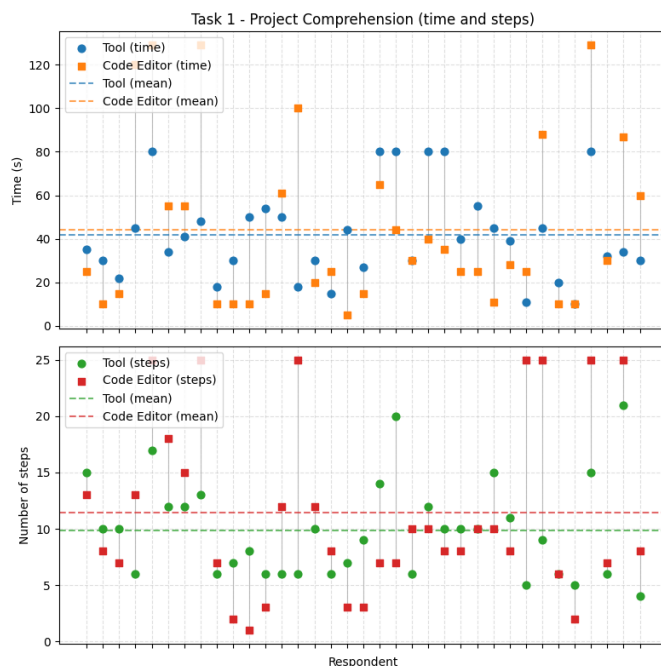
A. Orientácia v projekte:

Pre **H1** nástroj poskytuje podporu zvýraznením variabilných blokov kódu, ktoré pri umiestnení kurzora na ne zobrazia odkazy na súbory, v ktorých je daný blok referencovaný. To umožňuje plynulú navigáciu medzi súvisiacimi súborami bez opustenia aktuálneho kontextu. Okrem toho sú variabilné bloky kódu v nástroji vizuálne odlíšené, takže po načítaní súboru variabilné časti nesplývajú so základným kódom ako v bežnom editore. Pri manuálnom postupe bolo potrebné buď otvárať a prehliadať každý súbor samostatne, alebo použiť globálne celotextové vyhľadávanie, ktoré ale zobrazuje výsledky v samostatnom okne.

Z výsledkov testovania prvej úlohy vyšlo, že priemerný čas s nástrojom bol 41s a s manuálnym postupom 44s, teda rozdiel bol iba 3s v prospech nástroja. Priemerný počet krokov navigácie bol 10 v nástroji a 12 v bežnom vývojovom prostredí. Pri štatistickom teste hypotéza nebola potvrdená s hodnotou p_value 0,34 pri čase a 0,10 pri krokoch navigácie. V tomto konkrétnom prípade bola efektívnosť práce približne rovnaká, z časti za to bude môcť aj dostupnosť pokročilého celotextového vyhľadávania, ktoré umožňuje jednoducho vyhľadávať časti kódu podľa názvu dopytovanej variabilnej funkcionality. Priebeh testovania s meraniami jednotlivých účastníkov je zobrazený na Obr. 5.

B. Konfigurácia a odvodenie

V prípade **H2** sa konfigurácia variability vykonáva v bočnom paneli, kde sú prehľadne zobrazené a zoskupené všetky voliteľné funkcionality, premenné a obmedzenia. Pri otvorení súbore sú viditeľné iba aktivované funkcionality, vďaka čomu je aktuálny stav konfigurácie vždy jasný. Každý súbor umožňuje aj zobrazenie výsledného kódu, čo používateľovi umožňuje okamžite vidieť, ako bude súbor vyzerat' po odvodení. Pri manuálnom postupe neexistuje priamy spôsob odvodenia výrobku a jedinou možnosťou je manuálne odstránenie alebo zakomentovanie anotácií, čo sťažuje overenie aktuálnej konfigurácie projektu. Táto časť porovnania preto nemohla byť vyhodnotená úplne ekvivalentným spôsobom.



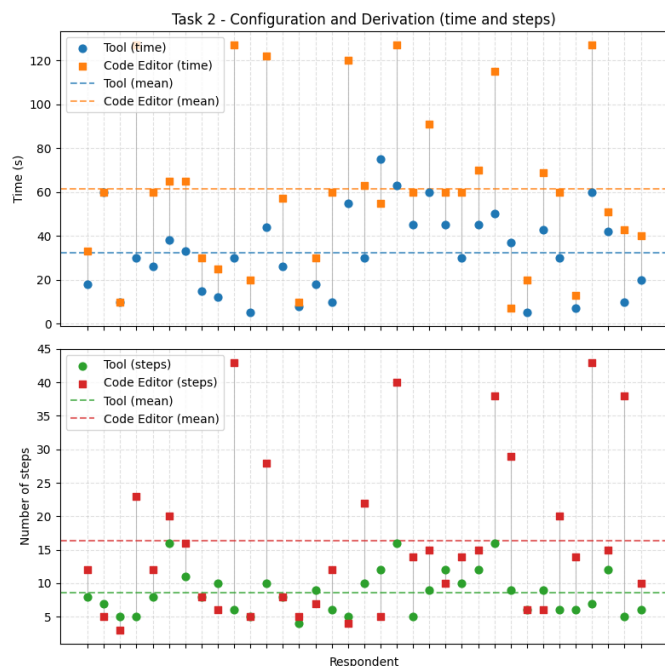
Obr. 5. Výkonnosť účastníkov testovania pri prvej úlohe

Počas používateľského testovania zmena hodnôt dvoch variabilných premenných s kontrolou obmedzení trvala 32s a vyžadovala 9 krokov v nástroji a 64s pri vykonaní 17 krokov v bežnom vývojovom prostredí. Obr. 6 zobrazuje priebeh testovania tejto úlohy. Hypotéza bola v tomto prípade potvrdená s hodnotou p_value menšou ako 0,001 v oboch prípadoch. To poukazuje na výrazné zlepšenie pri použití nástroja, zatiaľ čo manuálny postup stále zostáva náchylný na chyby, keďže kontrola obmedzení a overovanie konzistencie sú plne ponechané na používateľa bez akejkoľvek integrovanej validácie.

C. Úprava variabilného kódu

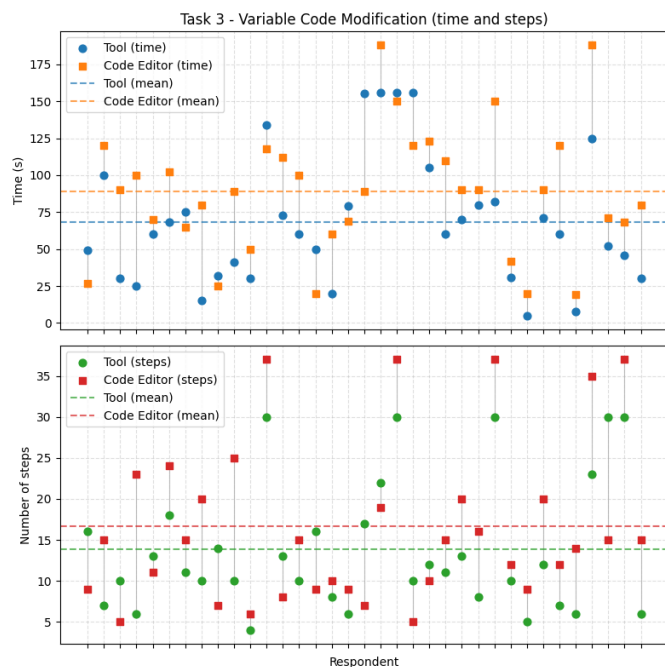
Počas testovania **H3** nástroj zaručoval, že pridanie novej variabilnej funkcionality v používateľskom rozhraní sa vo výsledku správne zabalí do požadovaných *XML* elementov. Taktiež pri definovaní hodnoty variabilnej premennej sa vytvorí korešpondujúci element v špecifikácii. V prípade manuálneho postupu boli vývojári zodpovední za správne vytvorenie *XML* elementov, a v prípade definovania nových hodnôt neboli zobrazované upozornenia na chýbajúce položky v konfigurácii, takže bolo vyžadované manuálne upraviť aj špecifikáciu projektu.

Podobne ako pri predchádzajúcej úlohe nástroj ukázal výrazne zvýšenú efektívnosť pri práci, s priemerným časom 68s a 14 krokmi. Práca v bežnom vývojovom prostredí v priemere trvala 88s a vyžadovala 17 krokov. Hypotéza bola potvrdená s p_value menšou ako 0,001 pri čase a 0,01 pri počte krokov. Vytváranie vlastných *XML* elementov ostáva náchylné na narušenie štruktúry projektu ako aj zachovanie konzistencie



Obr. 6. Výkonnosť účastníkov testovania pri druhej úlohe

medzi použitými a definovanými variabilnými premennými. Výsledky priebehu tejto úlohy sú zobrazené na Obr. 7.



Obr. 7. Výkonnosť účastníkov testovania pri tretej úlohe

D. Zhodnotenie

Pred začiatkom testovania účastníci poskytli aj demografické údaje týkajúce sa najmä ich vzdelania, zamestnania a skúseností v oblasti vývoja softvéru, či už všeobecne alebo

aj v zameraní na rady softvérových výrobkov. V dostupnej vzorke nebola preukázaná korelácia medzi skúsenosťami používateľov a efektívnosťou počas riešenia úloh. Pravdepodobne za to bude môcť aj strmá krivka učenia, na ktorú sa viacerí účastníci odvolávali v dodatočných otázkach. Celkové porovnanie výsledkov testovania hypotéz je zhrnuté v Tabuľke III.

Tabuľka III
ZHRNUTIE VÝSLEDKOV TESTOVANIA HYPOTÉZ

Úloha	Meranie	Nástroj	Editor	Rozdiel	Hypotéza
1	čas	41s	44s	3s	vyvrátená (0,3386)
1	kroky	9	11	2	vyvrátená (0,1039)
2	čas	32s	61s	29s	potvrdená (0,0000)
2	kroky	8	16	8	potvrdená (0,0002)
3	čas	68s	88s	20s	potvrdená (0,0004)
3	kroky	13	16	3	potvrdená (0,0134)

Po vykonaní testovacích scenárov boli účastníci vyzvaní na ohodnotenie ich spokojnosti s používaním nástroja na Likertových škálach (1 – úplne nesúhlasím, 5 – úplne súhlasím). V priemere hodnotenie dosahovalo úroveň 4, teda skôr súhlasím. Položené otázky spolu s ich priemerným a mediánovým hodnotením zobrazuje Tabuľka IV.

Tabuľka IV
VÝSLEDOK SUBJEKTÍVNEHO HODNOTENIA SPOKOJNOSTI

Otázka	Priemer	Medián
V nástroji som sa vedel/a rýchlo zorientovať	3,37	3
Nástroj mi uľahčil nájdenie relevantných častí kódu	4,00	4
Konfigurácia projektu bola v nástroji jednoduchá	4,09	4
Nástroj mi pomohol znížiť riziko chýb pri konfigurácii	4,00	4
Odvodenie výrobku bolo jednoduché a zrozumiteľné	4,17	4
Úprava variabilného kódu bola v nástroji prehľadná	3,94	4
Validácia a upozornenia boli užitočné	4,14	4
S nástrojom som bol/a celkovo spokojný/á	3,91	4

V. DISKUSIA

Aktuálna verzia nástroja nemá za cieľ nahradiť štandardné integrované vývojové prostredie (*IDE*) a mala by byť vnímaná iba ako podporné prostredie zamerané na manažment variability. Neposkytuje pokročilé vývojárske funkcionality a jazykovo špecifické vlastnosti, ktoré sú bežne dostupné v moderných *IDE*, a aj keď vynucuje syntax a pravidlá špecifické pre *LFP*, neobsahuje podporu pre programovací jazyk, v ktorom je implementovaný samotný rad softvérových výrobkov, preto nedokáže zaručiť, že výsledný kód je syntakticky správny. Nástroj takisto nedokáže zabrániť externým úpravám, ktoré do projektu môžu priniesť inkonzistencie v štruktúre alebo konfigurácii. Keďže nástroj je jazykovo nezávislý, je náročné analyzovať ako prierezové mechanizmy, v tomto prípade

AOP, ovplyvňujú funkcionálnosť kódu. Bez znalosti konkrétnej syntaxe AOP rozšírenia a konštruktov cieľového jazyka nie je možné deterministicky určiť, ktoré časti základného kódu budú daným aspektom ovplyvnené.

K zisteným technickým problémom patrí nedostupnosť *File System API* v prehliadačoch, ktoré nie sú založené na *Chromium* jadre, na čo bolo upozorňované počas používateľského testovania a pomerne dlhý počiatočný čas vykreslenia obrazovky, ktorý bol účastníkmi testovania nameraný v priemere na 1,45s pomocou nástroja *Lighthouse*. Ďalej sa nepodarilo presne vygenerovať model vlastností, pri ktorom sa objavili problémy s limitáciou knižnice *D3.js* pre tento účel. Aj keď knižnica dokáže správne zachytiť hierarchiu modulov a vnorených funkcionálností, nezodpovedá definícii a konvenciám modelu vlastností a má preto inú podobu.

Vývojári počas evaluácie upozornili na niekoľko problémov súvisiacich s použiteľnosťou nástroja. Väčšina účastníkov mala zo začiatku ťažkosti s pochopením toho, ako nástroj používať a orientovať sa v ňom a čo je jeho pridaná hodnota oproti bežnému *IDE*. Po dokončení všetkých úloh a oboznámení sa s používaním však svoju skúsenosť hodnotili prevažne pozitívne. Skúsenejší používatelia uviedli, že sa im lepšie pracuje s ich vlastne nastaveným vývojovým prostredím a nástroj videli užitočný skôr pre začiatočníkov.

Účastníci testovania navrhli aj konkrétne vylepšenia, ktoré by boli zohľadnené v rámci budúcej práce. Najrelevantnejšími z nich sú pridanie krátko interaktívneho dema pri prvom spustení nástroja, zobrazenie rozdielov zmien pred ich uložením s možnosťou potvrdenia alebo vrátenia, konfigurácia konkrétneho variabilného bloku priamo v editore bez nutnosti prechádzania do bočného panela a režim zobrazenia iba voliteľných častí kódu, v prípade úprav nesúvisiacich so spoločným kódom.

VI. ZÁVER

Práca analyzovala technológiu aspektov v rámci ako prístup k manažmentu variability v radoch softvérových výrobkov spolu s návrhom metodológie vývoja podporenej nástrojom pre XML anotované projekty s dôrazom na nezávislosť od programovacieho jazyka implementácie. Nástroj spracúva anotácie prevzaté z *LFP* technológie, uchováva internú reprezentáciu projektu a poskytuje podporné funkcionality vrátane správy konfigurácie variability, validácie a kontroly integrity a odvodenia výrobkov. Implementovaný je ako moderná SPA aplikácia, ktorá využíva technologický pokrok a umožňuje vyvíjať rady softvérových výrobkov pomocou aspektov v rámci.

Metodológia bola vyjadrená prostredníctvom konkrétnych metód pre orientáciu v projekte, konfiguráciu a odvodenie výrobkov a úpravu variabilného kódu. Navrhnuté aj existujúce funkcionality pre podporu manažmentu variability boli systematicky porovnané v tabuľkách, pričom bola zdôraznená kombinácia existujúcich mechanizmov pre správu variability, ako sú validácia XML, odvodzovanie výrobkov a generovanie dokumentácie, s navrhnutými funkciami, ako sú prepojenie a vizualizácia variabilných častí, dopytovanie bodov variability

pomocou CSS a zjednodušenie ukladania zmien v XML dokumentoch prostredníctvom používateľského rozhrania.

Porovnávacia štúdia bola realizovaná medzi navrhnutým nástrojom a manuálnym postupom v bežnom vývojovom prostredí. Výsledky ukázali, že navrhnutý prístup štatisticky významne zrýchlil konfiguráciu a odvodenie výrobkov a úpravu variabilného kódu, zatiaľ čo pri orientácii v projekte bol rozdiel nepreukázateľný. Vývojári po počiatočných ťažkostiach s používaním nástroja hodnotili ich skúsenosť prevažne pozitívne a na základe ich spätnej väzby boli identifikované kľúčové vylepšenia pre budúcu prácu. Celkovo výsledky naznačujú, že aspekty v rámci v kombinácii s modernými webovými technológiami spolu s ďalšími podpornými funkcionálitami môžu správu variability v radoch softvérových výrobkov spraviť praktickejšou a použiteľnejšou, obzvlášť v projektoch využívajúcich jazykovo nezávislé XML anotácie.

LITERATÚRA

- [1] P. Borba, L. Teixeira, and R. Gheyi, "A theory of software product line refinement," *Theoretical Computer Science*, vol. 455, pp. 2–30, 2012.
- [2] N. Loughran and A. Rashid, "Framed aspects: Supporting variability and configurability for aop," in *International Conference on Software Reuse*. Springer, 2004, pp. 127–140.
- [3] N. Loughran, A. Rashid, W. Zhang, and S. Jarzabek, "Supporting product line evolution with framed aspects," in *AOSD ACP4IS Workshop*, 2004.
- [4] M. Kassab and J. P. Fernandes, "The forgotten potential: Why aspect-oriented programming faded away," *Computer*, vol. 58, no. 3, pp. 100–104, 2025.
- [5] J. Perdek and V. Vranić, "Complexity of in-code variability: Emergence of detachable decorators," in *International Conference on Software and Software Reuse*. Springer, 2024, pp. 51–71.
- [6] T. Thüm, S. Apel, C. Kästner, M. Kuhlemann, I. Schaefer, and G. Saake, "Analysis strategies for software product lines," *School of Computer Science, University of Magdeburg, Tech. Rep. FIN-004-2012*, 2012.
- [7] J. Bosch, G. Florijn, D. Greefhorst, J. Kuusela, J. H. Obbink, and K. Pohl, "Variability issues in software product lines," in *International Workshop on Software Product-Family Engineering*. Springer, 2001, pp. 13–21.
- [8] C. Gacek and M. Anastasopoulos, "Implementing product line variabilities," in *Proceedings of the 2001 symposium on Software reusability: putting software reuse in context*, 2001, pp. 109–117.
- [9] M. Jaring and J. Bosch, "Representing variability in software product lines: A case study," in *International Conference on Software Product Lines*. Springer, 2002, pp. 15–36.
- [10] P. Sochos, I. Philippow, and M. Riebisch, "Feature-oriented development of software product lines: mapping feature models to the architecture," in *Net. ObjectDays: International Conference on Object-Oriented and Internet-Based Technologies, Concepts, and Applications for a Networked World*. Springer, 2004, pp. 138–152.
- [11] F. Heidenreich, J. Kopceck, and C. Wende, "Featuremapper: Mapping features to models," in *Companion of the 30th international conference on Software engineering*, 2008, pp. 943–944.
- [12] K. Lee, G. Botterweck, and S. Thiel, "Feature-modeling and aspect-oriented programming: Integration and automation," in *2009 10th acis international conference on software engineering, artificial intelligences, networking and parallel/distributed computing*. IEEE, 2009, pp. 186–191.
- [13] E. Figueiredo, N. Cacho, C. Sant'Anna, M. Monteiro, U. Kulesza, A. Garcia, S. Soares, F. Ferrari, S. Khan, F. Castor Filho et al., "Evolving software product lines with aspects: an empirical study on design stability," in *Proceedings of the 30th international conference on Software engineering*, 2008, pp. 261–270.
- [14] J. M. Horcas, M. Pinto, and L. Fuentes, "Empirical analysis of the tool support for software product lines," *Software and Systems Modeling*, vol. 22, no. 1, pp. 377–414, 2023.
- [15] M. El Dammagh and O. De Troyer, "Feature modeling tools: evaluation and lessons learned," in *International Conference on Conceptual Modeling*. Springer, 2011, pp. 120–129.

- [16] E. Kuitert. (2018) feature-schema. [Online]. Available: <https://github.com/ekuitert/feature-schema>
- [17] J. Meinicke, T. Thüm, R. Schröter, F. Benduhn, T. Leich, and G. Saake, *Mastering software variability with FeatureIDE*. Springer.
- [18] C. A. Welty, "Augmenting abstract syntax trees for program understanding," in *Proceedings 12th IEEE International Conference Automated Software Engineering*. IEEE, 1997, pp. 126–133.
- [19] M. F. S. Lazuardy and D. Anggraini, "Modern front end web architectures with react.js and next.js," *Research Journal of Advanced Engineering and Science*, vol. 7, no. 1, pp. 132–141, 2022.